

WorldEpisode: A Storage-Neutral Interchange Contract Linking Robot-Learning Episodes to Versioned, Replayable 3D Worlds

Alba Tellez Fernandez
URDF Studio
WorldEpisode draft maintainer

WorldEpisode Contributors

Abstract

Robot-learning datasets standardize observation–action trajectories, while 3D scene formats standardize visual and physical assets. However, the correspondence between an episode and the world in which it occurred remains encoded through dataset-specific assumptions, preventing reliable reconstruction, replay, conversion, counterfactual augmentation, and leakage-safe evaluation. We introduce *WorldEpisode*, a storage- and representation-neutral interchange profile that binds robot-learning episodes to immutable world revisions through explicit clocks and coordinate frames, persistent entity identity, fully specified action semantics, multi-representation asset roles, interaction events, provenance, and uncertainty. WorldEpisode does not replace existing containers or scene formats; it provides bindings for LeRobotDataset, Rerun, NCore, MCAP, OpenUSD, glTF Gaussian splats, and simulator-specific world targets. We outline an executable conformance suite, loss-aware converters, and a benchmark plan that evaluates semantic round trips, physical-coherence fault detection, cross-simulator replay, and lineage-safe dataset splitting. Gaussian splats are a high-value appearance profile, but the central contribution is persistent, verifiable world–episode interoperability.

1 Introduction

Robot-learning datasets and 3D scene formats are improving quickly, but they solve different problems. LeRobot-style datasets focus on efficient observation–action episodes. Rerun provides a physical-data layer and storage target for multimodal recordings. NCore documents explicit pose graphs and sensor conventions. OpenUSD, SimReady, glTF, and simulator-native formats describe composed worlds and assets. Gaussian splats are also moving into standard asset containers through glTF’s `KHR_gaussian_splatting` extension [5, 6]. Recent real-to-sim systems such as RoboSnap and DROID-Sim show that Gaussian-splat environments can support trajectory replay, synthetic data generation, and policy evaluation at dataset scale [4].

This landscape makes one framing unsafe: WorldEpisode should not claim to be the first format that connects robot data and Gaussian-splat worlds. That space is already occupied by asset standards and method papers. The remaining opening is the contract connecting all of them: the persistent, auditable binding between a robot-learning episode and the versioned world in which it occurred.

WorldEpisode targets that missing semantic layer. It is:

- **storage-neutral**: serializable through LeRobot, Rerun, NCore, MCAP, or a reference package;
- **representation-neutral**: compatible with splats, meshes, NeRFs, point clouds, and future assets;

- **runtime-neutral**: transferable to Isaac Sim, MuJoCo, SAPIEN, Genesis, and other simulators;
- **loss-explicit**: conversions may be lossy, but never silently lossy.

The paper’s core claim is therefore not a new monolithic file format. It is an executable interchange profile for world–episode semantics: identity, frames, clocks, action contracts, representation roles, events, provenance, uncertainty, and lineage-safe evaluation splits.

2 Problem Statement and Scope

WorldEpisode binds an episode to a world revision. The base state is:

$$W_e(t) = W^{(r)} \oplus \Delta W_e(t), \quad (1)$$

where $W^{(r)}$ is an immutable, content-addressed world revision, $\Delta W_e(t)$ is the ordered sequence of episode-specific state changes, e is a robot-learning episode, and t is expressed in an explicitly declared clock domain.

An episode is modeled as:

$$E = \langle M, W^{(r)}, B, K, F, C, O, A, X, V, P, Q \rangle, \quad (2)$$

where M is the manifest, B the embodiment, K task/outcome semantics, F the frame graph, C clocks, O persistent entities and representations, A action-space contract, X synchronized trace, V world deltas and events, P provenance, and Q quality and uncertainty records.

Version 1 should be intentionally narrow: rigid tabletop manipulation with fixed-base single- or dual-arm robots. Humanoids, locomotion, deformables, fluids, and multi-agent worlds should be later profiles. This keeps conformance executable rather than aspirational.

WorldEpisode should reuse existing standards rather than replacing them. USD [1] can carry composed simulation worlds, glTF can carry Gaussian appearance assets [5], NCore can carry capture and pose-graph conventions [2], Rerun can carry physical-data recordings with column chunks [3], and ROS/MCAP can carry logs. WorldEpisode specifies the semantic relationships those bindings must preserve.

3 The Five-Graph Model

3.1 Identity Graph

Every physical or logical entity receives a persistent identifier: robot, link, gripper, sensor, rigid object, articulated object, fixture, support surface, region, human, light, or semantic group. The same `entity_id` must be used by segmentation masks, bounding boxes, Gaussian groups, render meshes, collision bodies, simulator actors, language references, contact events, object trajectories, grasp events, and attachment events.

3.2 Frame and Clock Graph

Every spatial value declares source frame, target frame, units, handedness, up axis, transform direction, quaternion convention, validity interval, calibration revision, and uncertainty where available. Every stream declares a clock domain, device timestamp, host-receive timestamp where available, exposure interval for cameras, effective command time for actions, synchronization mapping, and estimated offset/drift/error. NCore’s explicit pose-graph convention demonstrates the kind of rigor WorldEpisode should require across the whole episode [2].

3.3 Representation-Role Graph

One physical entity can have many representations: Gaussian splat appearance, textured render mesh, metric surface mesh, depth point cloud, collision proxy, mass/inertia record, semantic affordance, or learned embedding. Each representation declares its role and an asset descriptor: URI, media type, SHA-256 digest, optional mirrors, optional embedded payload metadata, coordinate frame, units, validity interval, derivation, uncertainty, and license. The URI may be relative, HTTP(S), Hugging Face, object storage, OCI, IPFS, or another registered resolver. Portability comes from deterministic resolution and digest verification, not from requiring every asset to live in one folder. The role field is essential: a splat may be valid for appearance but invalid for collision.

3.4 Temporal State and Event Graph

Episodes should not store only poses. They should record interaction events: contact begin/end, grasp establish/release, attachment create/remove, support, containment, articulation state changes, spawn/despawn, topology changes, simulator reset, intervention, subgoal completion, and failure detection. Each event references entities, time, confidence, source, and optionally evidence.

3.5 Provenance and Derivation Graph

Every derived asset should identify source episodes, source sensors and intervals, reconstruction algorithm, software version, model/checkpoint hash, configuration, calibration revision, manual edits, source and output hashes, creator, license, and quality metrics. WorldEpisode should map to existing provenance and dataset metadata vocabularies rather than inventing an incompatible publication vocabulary.

4 Action Semantics, Revisions, and Loss Reports

4.1 Complete Action Semantics

An action vector is not portable without its operational contract. Each action channel must declare actuator, control mode, parameterization, reference frame, units, absolute/delta/velocity semantics, limits, command rate, command timestamp, effective timestamp, latency model, interpolation, normalization, gripper semantics, and missing-value policy. A vector such as $[0.01, 0, 0, 0, 0, 0, 1]$ is otherwise ambiguous: it could be a tool-frame delta, base-frame delta, velocity command, normalized policy output, or delayed target increment.

4.2 Immutable World Revisions

Each episode references one base world revision plus ordered episode deltas. A new revision is required when world geometry, metric scale, calibration, collision proxies, physical parameters, entity decomposition, object identity, or task-relevant fixtures change. Content hashes allow two datasets to prove that they refer to the same world without relying on filenames.

4.3 Loss-Aware Conversion

WorldEpisode should support many bindings, but it should not promise impossible universal losslessness. Every converter emits a machine-readable report listing preserved, externalized, approximated, discarded, and warning fields. The standard permits lossy conversion; it forbids silent lossy conversion.

4.4 Reference SDK

The reference release should feel like infrastructure:

```
worldepisode validate dataset/  
worldepisode inspect dataset/  
worldepisode diff-world world_a world_b  
worldepisode audit-splits dataset/  
worldepisode report-lineage dataset/  
worldepisode replay-check episode_id  
worldepisode import lerobot ...  
worldepisode import ncore ...  
worldepisode import mcap ...  
worldepisode import gsdf ...  
worldepisode export lerobot ...  
worldepisode export rerun ...  
worldepisode export usd ...  
worldepisode export gltf ...  
worldepisode conversion-report report.json
```

The validator should use stable requirement identifiers, profile claims, and machine-readable diagnostics. Golden fixtures should include valid minimal packages, valid advanced packages, intentionally invalid packages, lossy-conversion examples, and cross-simulator replay fixtures. URDF Studio can remain the initial validation and simulator-transfer workbench, but the contract must be implementable independently.

5 Conformance and Evaluation

WorldEpisode should be evaluated as an executable standard, not only as a schema.

RQ1: semantic preservation across bindings. Convert the same dataset through LeRobot, WorldEpisode, Rerun, NCore, MCAP, OpenUSD, and glTF Gaussian assets. Measure field retention, semantic retention, deterministic round-trip equality, externalized fields, approximated fields, and declared loss.

RQ2: physical-coherence fault detection. Inject controlled failures: wrong transform direction, millimeters interpreted as meters, quaternion-order mismatch, timestamp offset, stale calibration, action frame mislabeled, collision mesh scale mismatch, reused entity id, or world revision changed without a new hash. Measure validator precision, recall, and diagnosis quality.

RQ3: cross-simulator replay. Replay the same demonstrations in at least two simulators. Measure end-effector RMSE, object trajectory RMSE, grasp-state agreement, contact-event precision/recall/F1, final-state pose error, task-outcome agreement, and success-rate rank correlation across policy checkpoints.

RQ4: downstream robustness. Train with observations/actions alone, with unstructured 3D side files, and with full WorldEpisode identity/world alignment. Evaluate shifted camera extrinsics, unseen viewpoints, object displacement, background replacement, distractors, occlusion, lighting changes, novel object instances, and held-out worlds.

RQ5: lineage-safe splits. Compare random episode splits against task-, world-, entity-, and reconstruction-lineage-disjoint splits. A substantial performance drop under lineage-safe splits would show that current dataset abstractions overestimate generalization.

The minimum credible reference release should include a small golden conformance corpus: valid minimal packages, valid advanced packages, intentionally invalid packages, lossy-conversion examples, and cross-simulator replay fixtures.

6 Discussion

Not another monolithic binary. WorldEpisode should have a small semantic core and multiple bindings. LeRobot can be the policy-training view; Rerun the multirate physical-data view; NCore the sensor/reconstruction capture view; MCAP/ROS the raw log view; OpenUSD/SimReady the composed simulation-world view; glTF or SPZ the Gaussian asset view; and a reference directory the conformance profile.

Lineage-safe evaluation. Episode-level random splits can leak the same reconstructed room, physical object, Gaussian asset, source video, world revision, or generated asset lineage across training and evaluation. WorldEpisode should support split constraints such as `world_lineage`, `physical_entity`, `source_capture`, `reconstruction_run`, and `generated_asset_family`.

Standards alignment. The project should align with standards bodies rather than antagonize them. Static 3D robot-map exchange, humanoid dataset lifecycle, OpenLABEL annotation, glTF Gaussian splats, USD composition, and NCore capture conventions are all useful neighbors. WorldEpisode is an executable profile and bridge across those layers.

Reviewer red team. Likely criticisms are predictable: “this is just USD,” “GSDF already combines splats and physics,” “Rerun already solves robot data,” or “a schema is not research.” The response is to ship adapters, conformance fixtures, fault-injection evaluation, replay metrics, and lineage-split evidence rather than relying on schema novelty alone.

7 Conclusion

WorldEpisode reframes the opportunity away from “LeRobot plus Gaussian splats in one file” and toward persistent, verifiable world-episode interoperability. Gaussian splats remain a high-value profile, but the durable contribution is a storage-neutral contract for immutable world revisions, persistent entity identity, frame and clock graphs, action semantics, representation roles, events, provenance, uncertainty, loss-aware conversion, and lineage-safe evaluation. The immediate next step is to harden the v0.1 schema, release a validator and golden conformance corpus, and demonstrate independent bindings across existing robot-data and 3D-world systems.

References

- [1] Alliance for OpenUSD. OpenUSD: Universal scene description. <https://openusd.org/>, 2026. Accessed 2026-07-13.
- [2] NVIDIA. NCore Specification: Data Conventions. <https://nvidia.github.io/ncore/data/conventions>, 2026. Accessed 2026-07-13.

- [3] Rerun. A new data layer for robot learning. <https://rerun.io/blog/data-layer-for-robot-learning>, 2026. Accessed 2026-07-13.
- [4] RoboSnap Authors. RoboSnap: One-Shot Real-to-Sim Scene Generation for Generalizable Robot Learning and Evaluation. <https://arxiv.org/html/2607.06699v1>, 2026. Accessed 2026-07-13.
- [5] The Khronos Group. KHR_gaussian_splatting. https://github.com/KhronosGroup/glTF/blob/main/extensions/2.0/Khronos/KHR_gaussian_splatting/README.md, 2026. Accessed 2026-07-13.
- [6] The Khronos Group. Khronos Announces glTF Gaussian Splatting Extension. <https://www.khronos.org/news/press/glTF-gaussian-splatting-press-release>, 2026. Accessed 2026-07-13.